



UNIVERSIDAD
DE GRANADA

Departamento de Ciencias de la
Computación e Inteligencia Artificial

Diseño del TDA “Juego de la Vida”

Estructuras de Datos

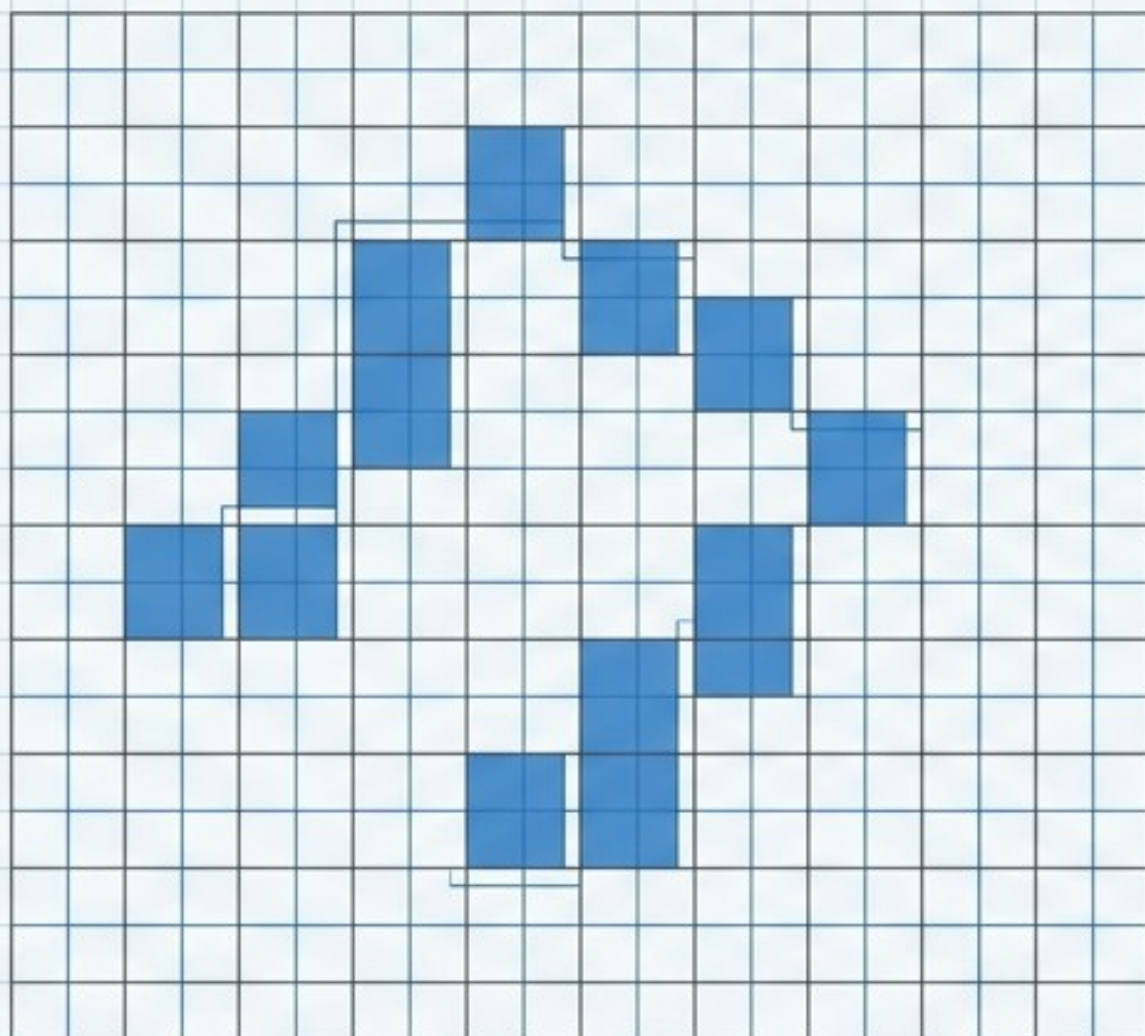
Grado en Ingeniería Informática

Doble Grado en Ingeniería Informática y Matemáticas | Doble Grado en Ingeniería Informática y ADE

Reto 1: Abstracción

Dpto. Ciencias de la Computación e Inteligencia Artificial
E.T.S. de Ingenierías Informática y de Telecomunicación
Universidad de Granada

Introducción al Autómata Celular

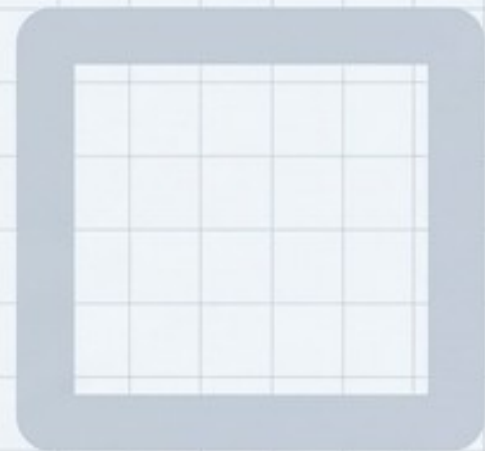


El Juego de la Vida (Conway's Life) es el ejemplo más famoso de autómata celular y fue diseñado por el matemático J. H. Conway en 1970. Una breve descripción sería:

- El "universo" consiste en una matriz bidimensional rectangular de celdas.
- Cada celda puede encontrarse, en un momento dado, en el estado viva (1) o muerta (0).

Como se puede apreciar en realidad el juego de la vida no es un "juego" en absoluto, ya que dado un patrón inicial, su evolución está completamente prefijada y no requiere de más datos provenientes del usuario.

Muerta (0)



Viva (1)



Exactamente 3 vecinos vivos

Nace

2 o 3 vecinos vivos

Sobrevive

< 2 o > 3 vecinos vivos

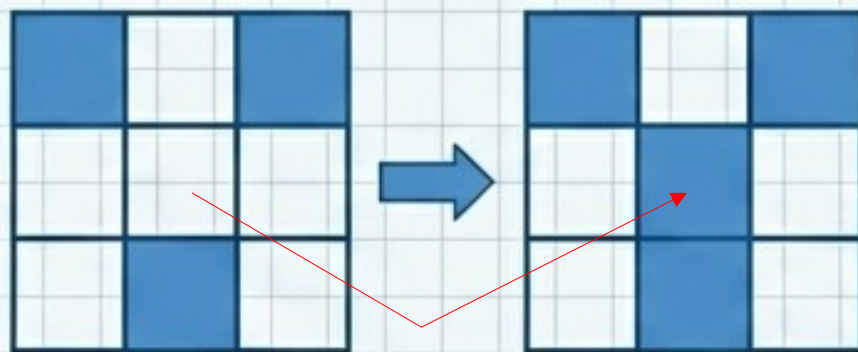
Muere por soledad o sobrepoblación

Nota: En cualquier otro caso, el estado actual se mantiene inalterado.

Mecánica de Transición: Las Reglas

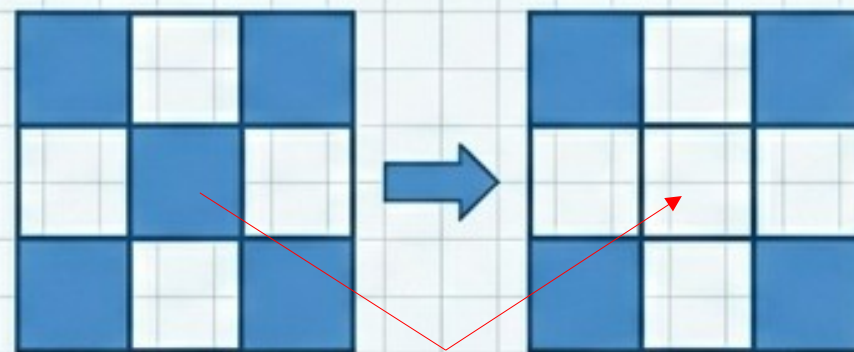
En cada nueva iteración, cada celda puede cambiar de estado dependiendo de cuantas de sus 8 celdas vecinas están vivas:

Nacimiento



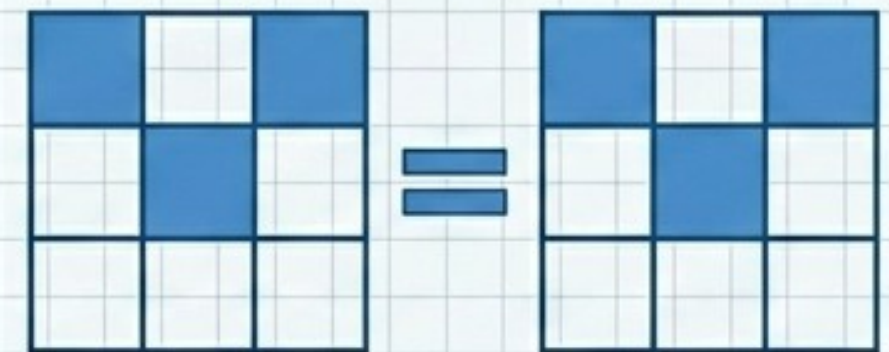
- Si una celda muerta está rodeada exactamente por 3 celdas vivas, pasa al estado viva.

Muerte



- Si una celda viva no está rodeada por 2 o 3 celdas vivas, pasa al estado muerta.

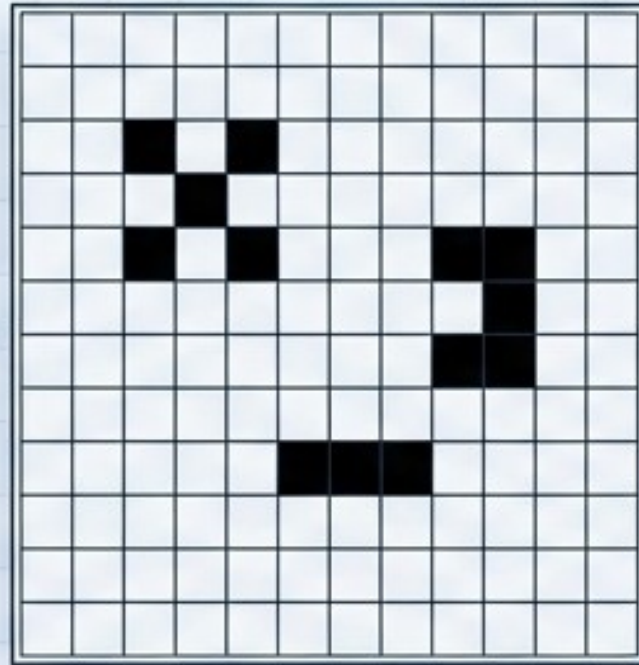
Estabilidad



- En cualquier otro caso la celda no cambia de estado.

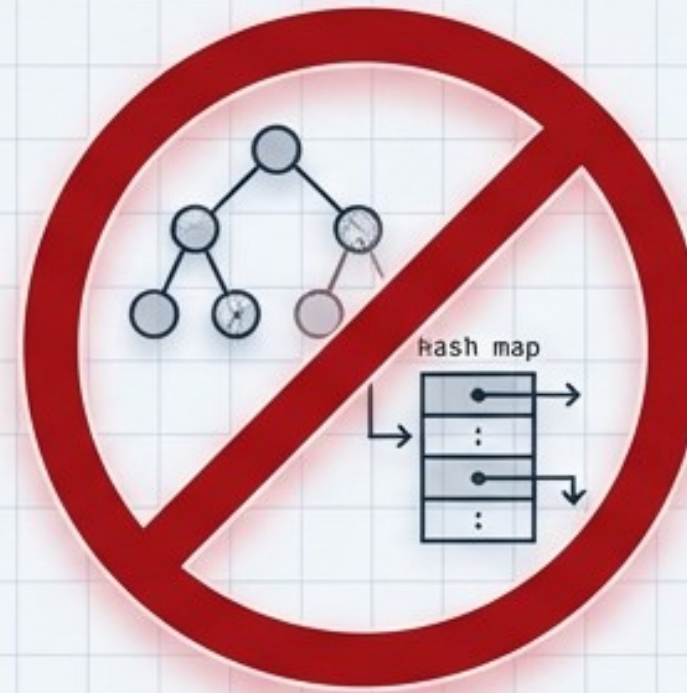
El Fenómeno Matemático

●
1970: J.H. Conway



Evolución determinista de patrones en una matriz rectangular. Cada celda es binaria: viva (1) o muerta (0). No requiere interacción del usuario post-inicialización.

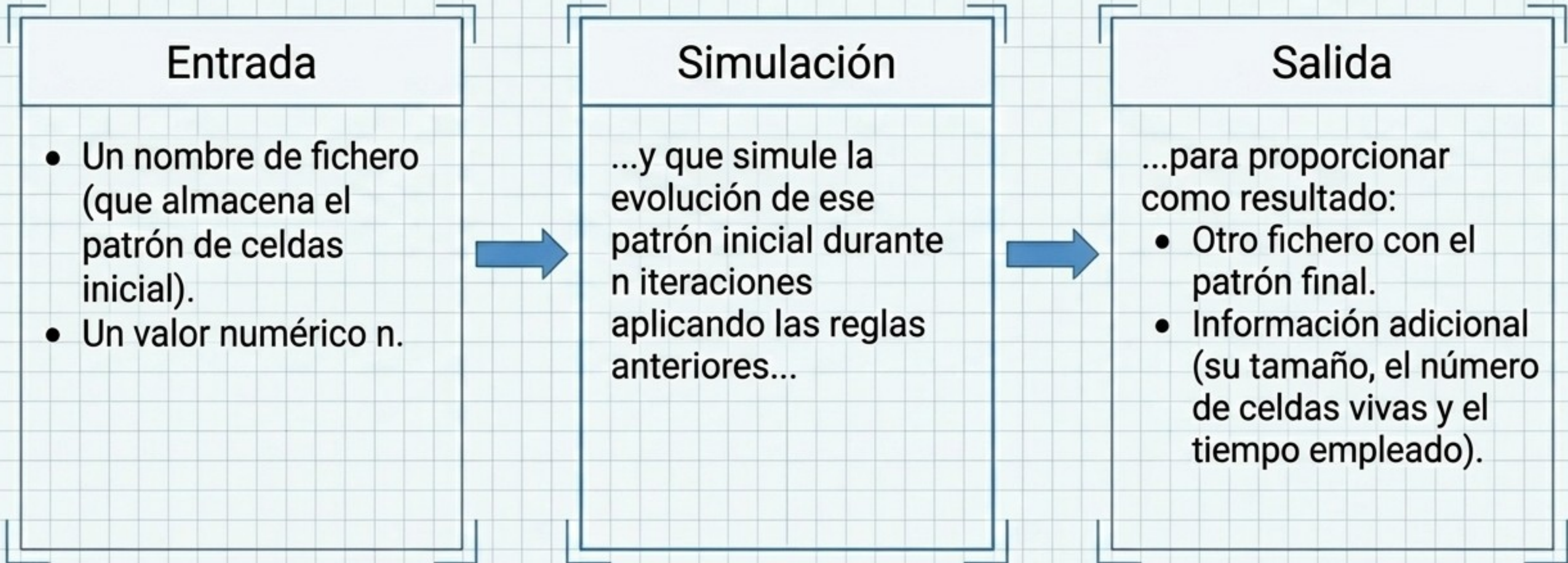
El Desafío de Abstracción



Diseñar una solución lo más eficiente posible en tiempo y espacio utilizando exclusivamente **vectores o matrices** rectangulares. Prohibido el uso de estructuras de datos avanzadas. **Cero líneas de código** implementado: solo especificación.

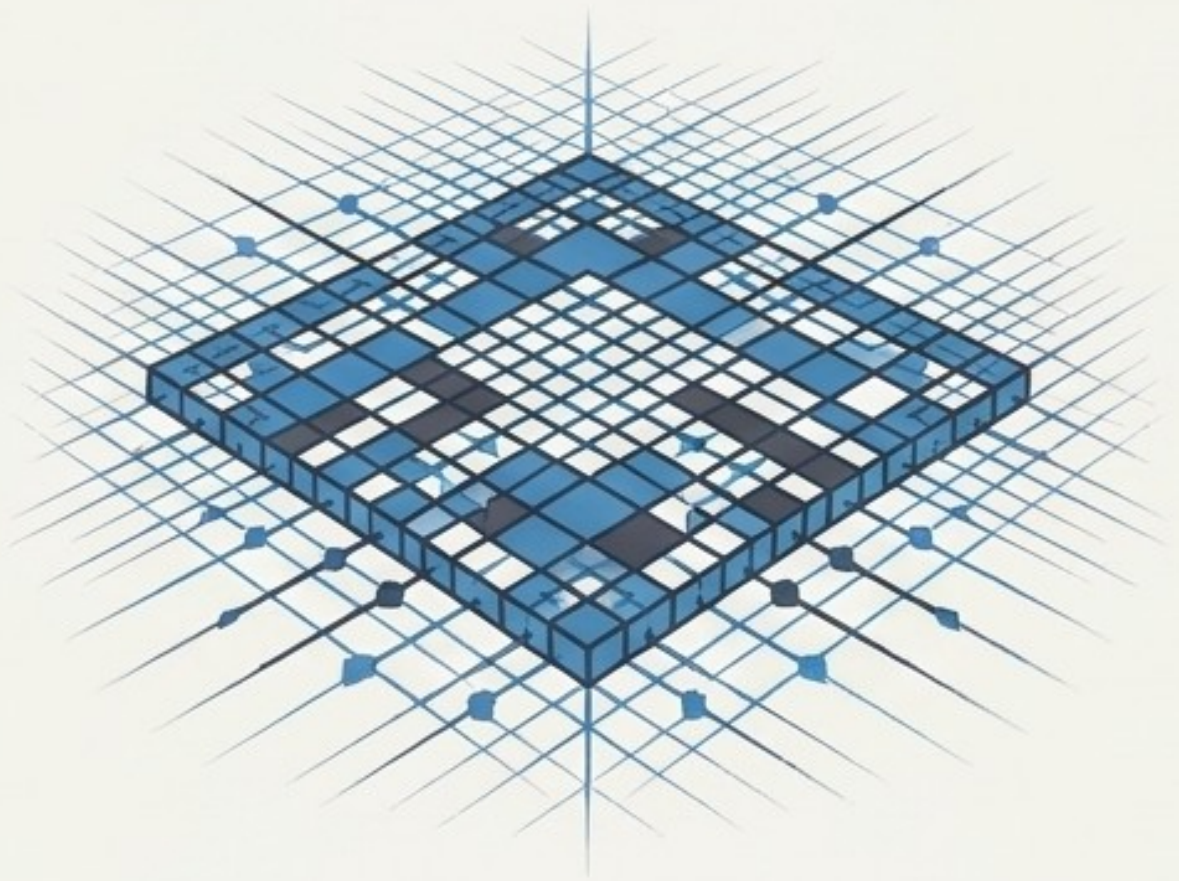
El Objetivo: Abstracción y Simulación

El objetivo del reto es hacer una abstracción sobre el problema que conduzca eventualmente a un programa al que se le indique:



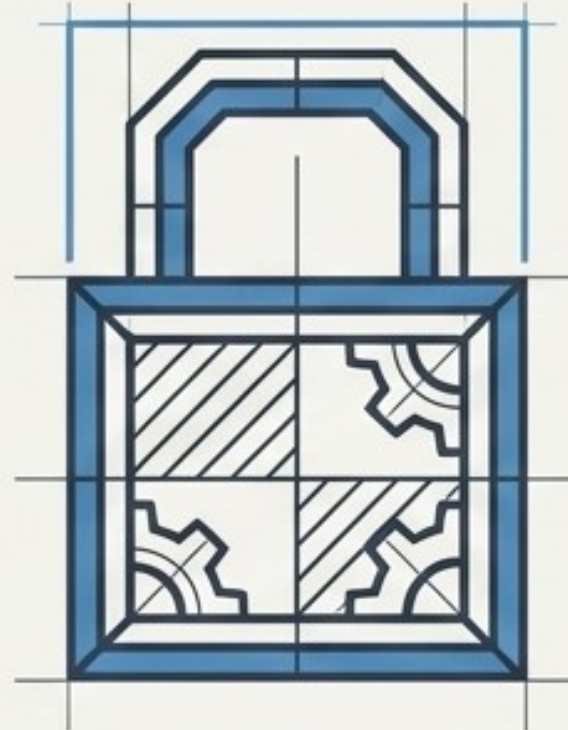
Restricciones Técnicas y de Diseño

El Universo Infinito



El tamaño de la matriz donde evolucionan los patrones no está limitado (es decir, que se puede ampliar dinámicamente para dar cabida a todas las celdas vivas que se generen durante la evolución del patrón inicial).

Eficiencia y Estructuras Permitidas



La abstracción debe tener en cuenta que el programa que se construya a partir de ella debe ser lo más eficiente posible en cuanto a tiempo y espacio, pero con la restricción de que:

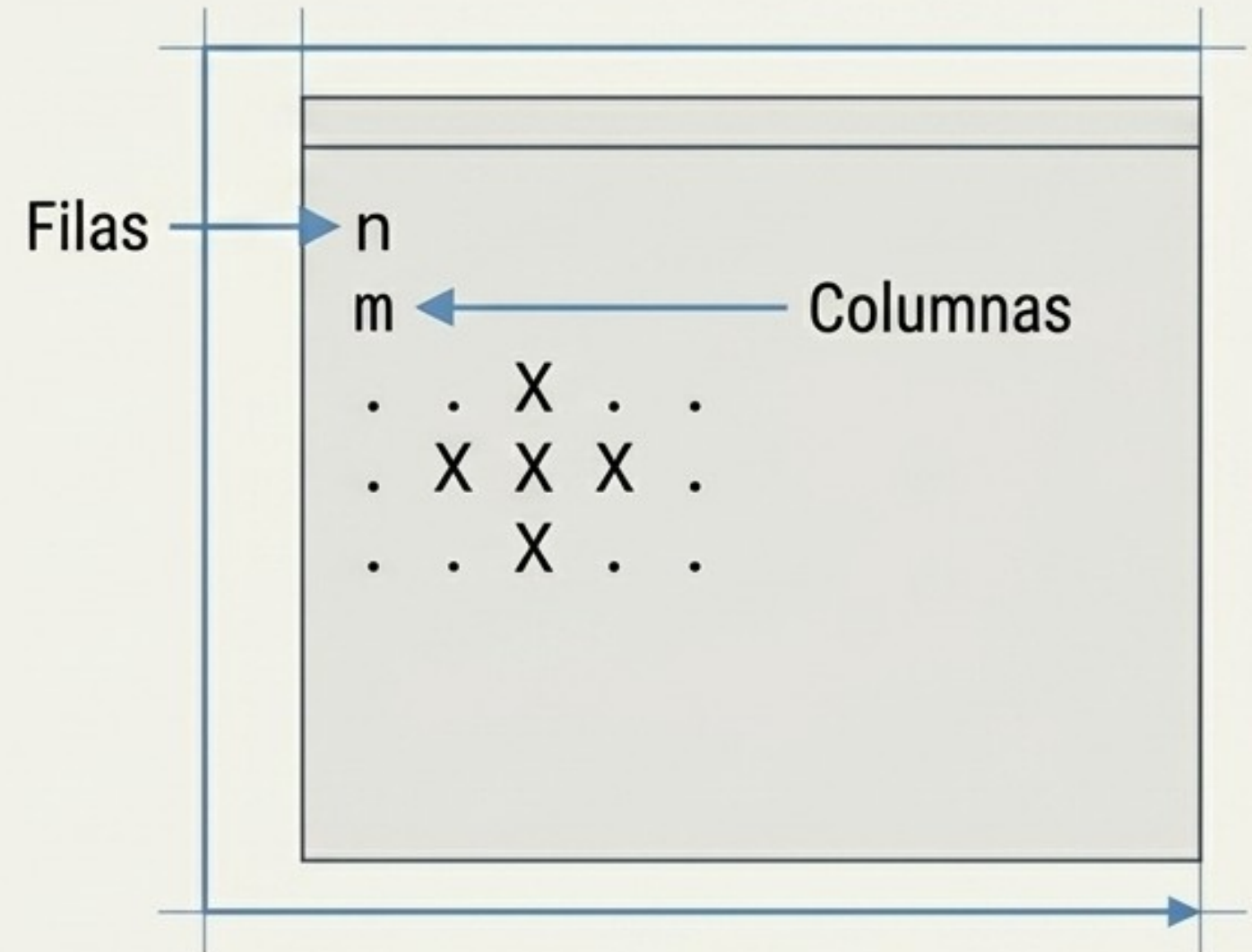
- Debe basarse **EXCLUSIVAMENTE** en el uso de vectores ó matrices rectangulares.
- **NO** emplear estructuras de datos o algoritmos avanzados (hashing, árboles, etc.).

Descripción Detallada: Entrada de Datos

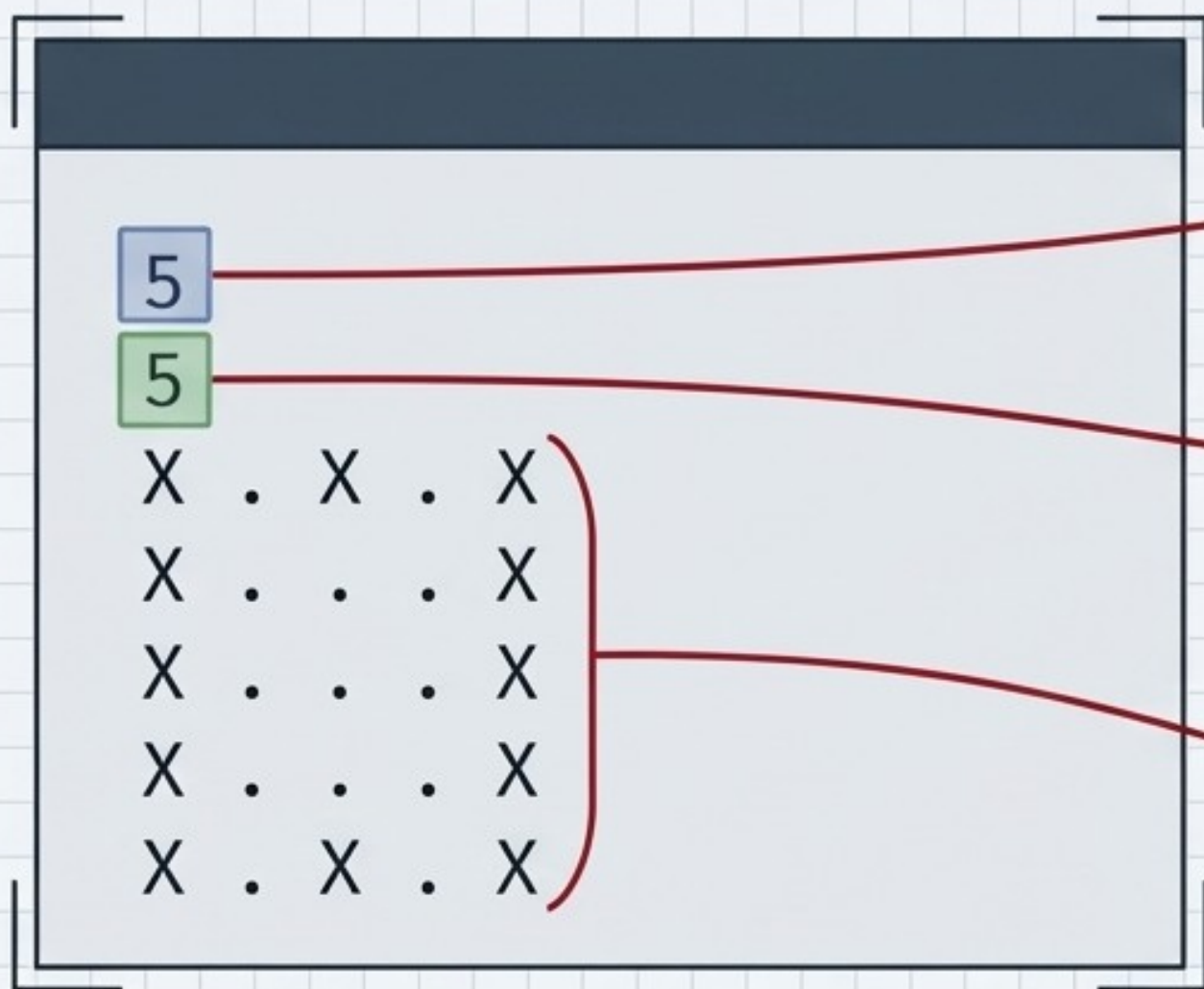
Como se ha indicado, el programa resultante de la abstracción recibe como entrada un nombre de fichero y un entero que indica el número de iteraciones. Estas entradas pueden pedirse al usuario (ya sea en modo consola o gráfico) o proporcionarse como parámetros del programa.

El nombre del fichero debe hacer referencia a un fichero de texto existente que almacena un rectángulo de celdas (denominado patrón) y que tiene el siguiente formato:

- La primera línea contiene un entero que indica el número de filas del patrón, n.
- La segunda línea contiene un entero que indica el número de columnas del patrón, m.
- Le siguen n líneas de texto de m caracteres cada una, los caracteres pueden ser el punto (celda muerta) o una X mayúscula (celda viva).



Anatomía del Formato de Intercambio (.txt)



Variable *n*
(Filas del patrón inicial)

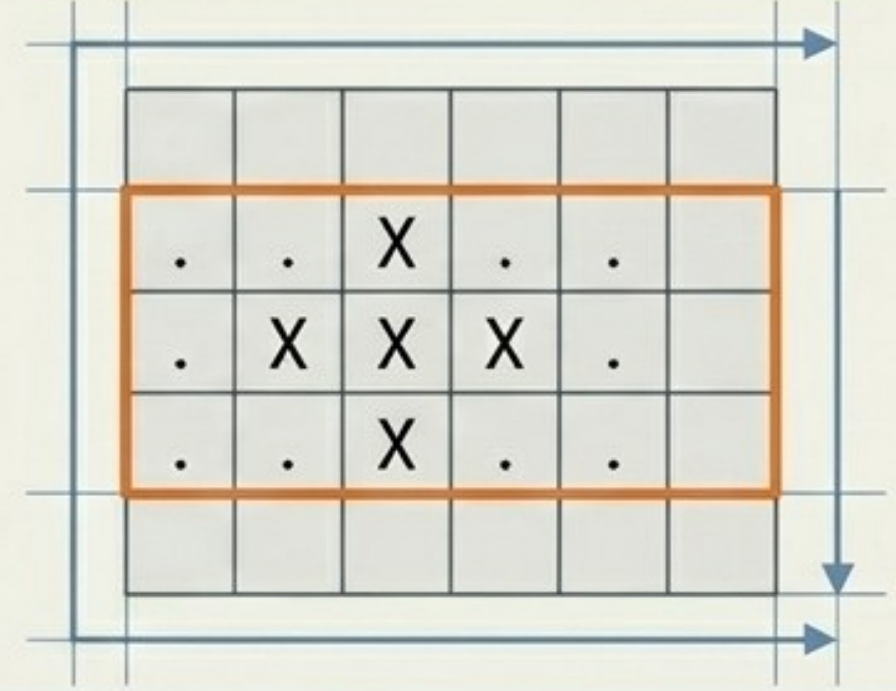
Variable *m*
(Columnas del patrón inicial)

Representación del estado:
'.' = 0 (Muerta), 'X' = 1 (Viva)

La lectura y escritura debe respetar este contrato de formato estrictamente para validar la persistencia del estado.

Rectángulo Mínimo y Crecimiento

Un patrón siempre se indica mediante el rectángulo más pequeño que contiene a todas las celdas vivas. Nunca se puede dar el caso de que la primera o última fila, o la primera o última columna estén totalmente formadas por celdas muertas. Atención porque la salida del programa (el patrón final) debe cumplir esta condición.



Crecimiento

Tras recibir estos datos, el programa pasaría a simular el número de iteraciones indicadas aplicando las reglas del juego de la vida. Hay que tener en cuenta que el espacio necesario puede tener que incrementarse a medida que transcurre la simulación.

Optimizaciones

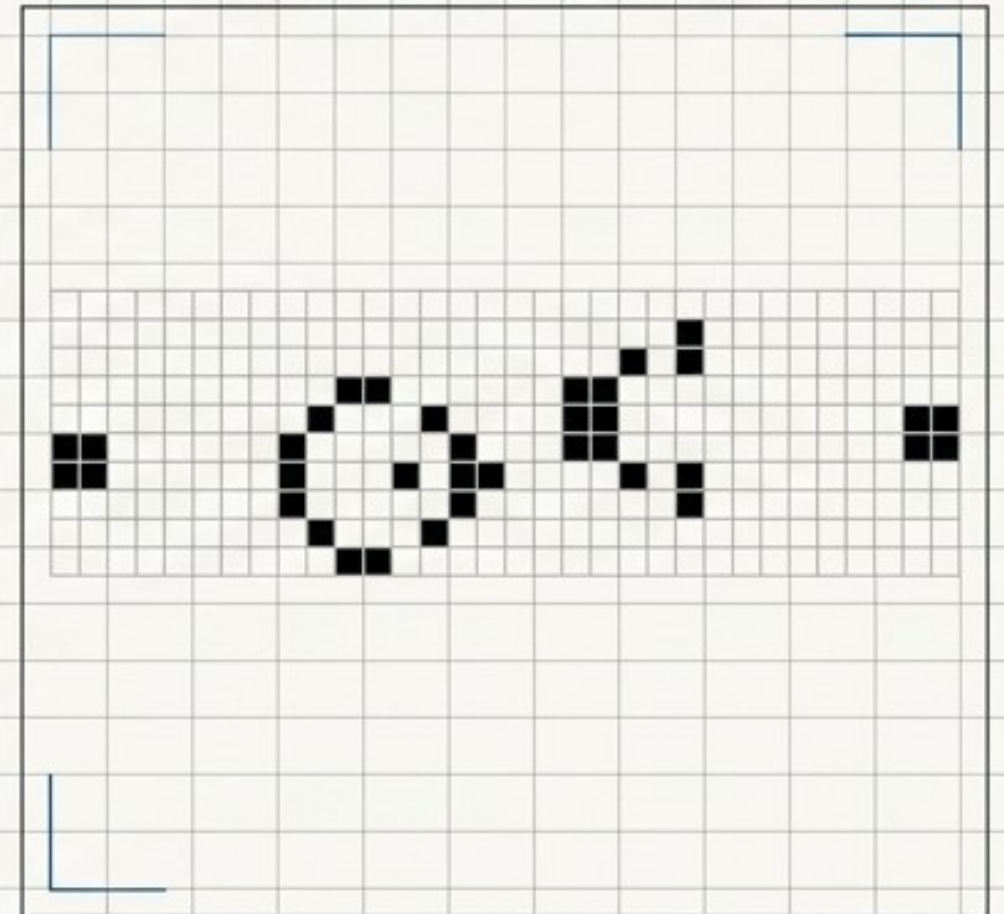
Se pueden aplicar optimizaciones sencillas para ahorrar espacio (por ejemplo, reutilizar o tener en cuenta las filas o columnas de los bordes de la matriz que tras una iteración pasen a constar únicamente de celdas muertas) pero no se pueden usar representaciones comprimidas de la matriz u otros métodos avanzados.

Ejemplo de Crecimiento: Cañón de Deslizadores

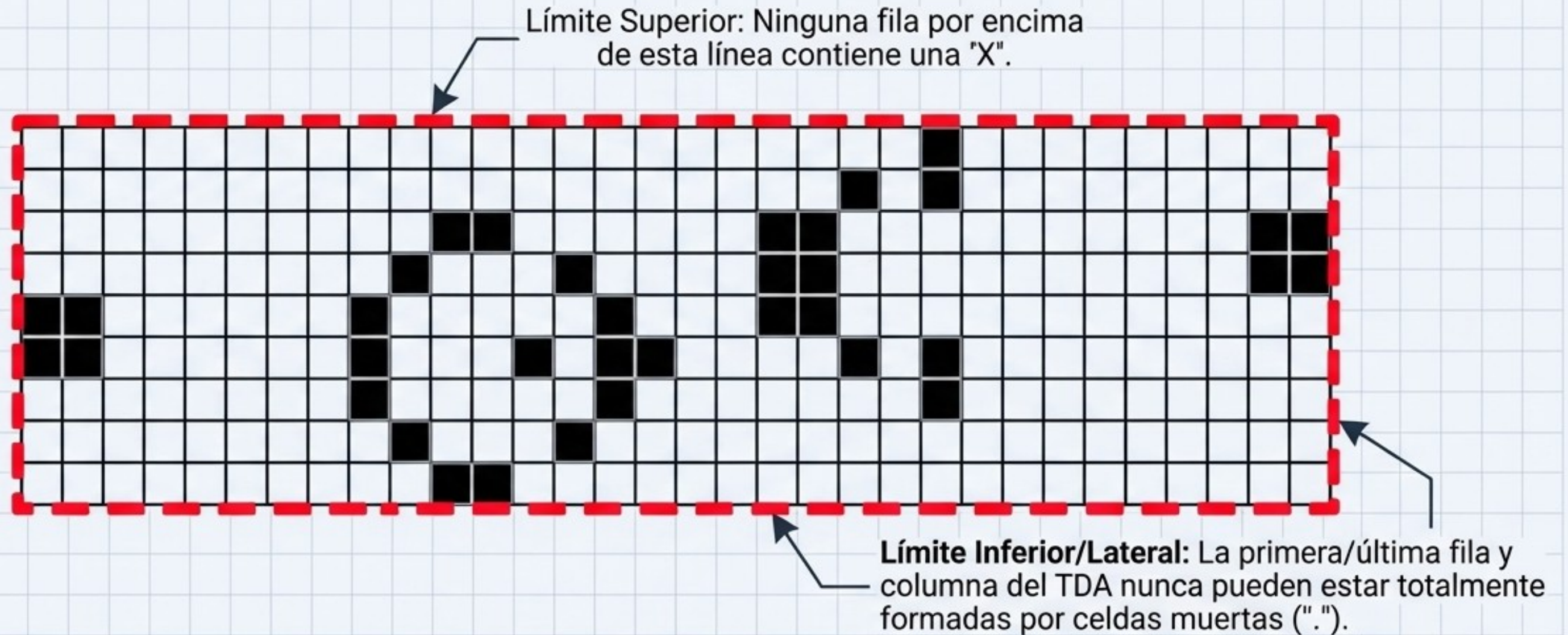
Por ejemplo, el patrón **cañón de deslizadores** de Gosper genera deslizadores (patrones que se desplazan) y aunque en las etapas iniciales ocupa únicamente una matriz de 8x36 celdas, a partir de un determinado momento cada 4 iteraciones necesita una fila y columna adicional para representar todas las celdas vivas.

Se representaría mediante el siguiente fichero de texto:

```
9
36
.....X.....
.....X.X.....
.....XX.....XX.....XX
.....X...X...XX.....XX
XX.....X...X...XX.....
XX.....X..X.XX...X.X.....
.....X....X....X.....
.....X..X.....
.....XX.....
```



La Regla de la Frontera Estricta (Rectángulo Mínimo)



El patrón siempre se define por el rectángulo más pequeño que contiene todas las celdas vivas. El TDA debe garantizar esta postcondición en cada salida.

Finalización y Resultados

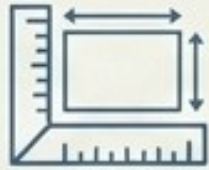
Al finalizar la simulación, el programa debería mostrar los siguientes datos: (el orden debe ser exactamente el indicado y cada ítem ocupar una línea distinta)



- Número de iteraciones realizadas



- Número de celdas vivas del patrón final



- Dimensiones (número de filas y columnas) del rectángulo mínimo que contiene a todas las celdas vivas del patrón final.



- Tiempo empleado por el programa.

Por último, pedirá al usuario el nombre del fichero donde se almacenará el patrón final (con el formato descrito anteriormente). Si el usuario introduce una cadena vacía no se almacenará el resultado.

Ejemplo Completo: El Patrón "Exploder" (Entrada y Ejecución)

Si el fichero `exploder.txt` contiene el siguiente patrón:

```
5
5
X.X.X
X...X
X...X
X...X
X.X.X
```

Un ejemplo de la ejecución del programa podría ser la siguiente (en cursiva datos del usuario):

```
> ./reto1 exploder.txt 100
100 iteraciones
72 celdas vivas
Dimensiones: 13 x 13
0.03 segundos
Fichero de salida: resultado.txt
```

Ejemplo Completo: El Patrón “Exploder” **(Salida)**

Y el contenido del fichero resultado.txt sería:

```
13
13
..XX      XX..
...XX...XX...
X..X.X.X.X..X
XXX.XX.XX.XXX
.X.X.X.X.X.X.
..XXX...XXX..
.....
..XXX...XXX..
.X.X.X.X.X.X.
XXX.XX.XX.XXX
X..X.X.X.X..X
...XX...XX...
..XX      XX..
```

SE PIDE: Especificación del TDA

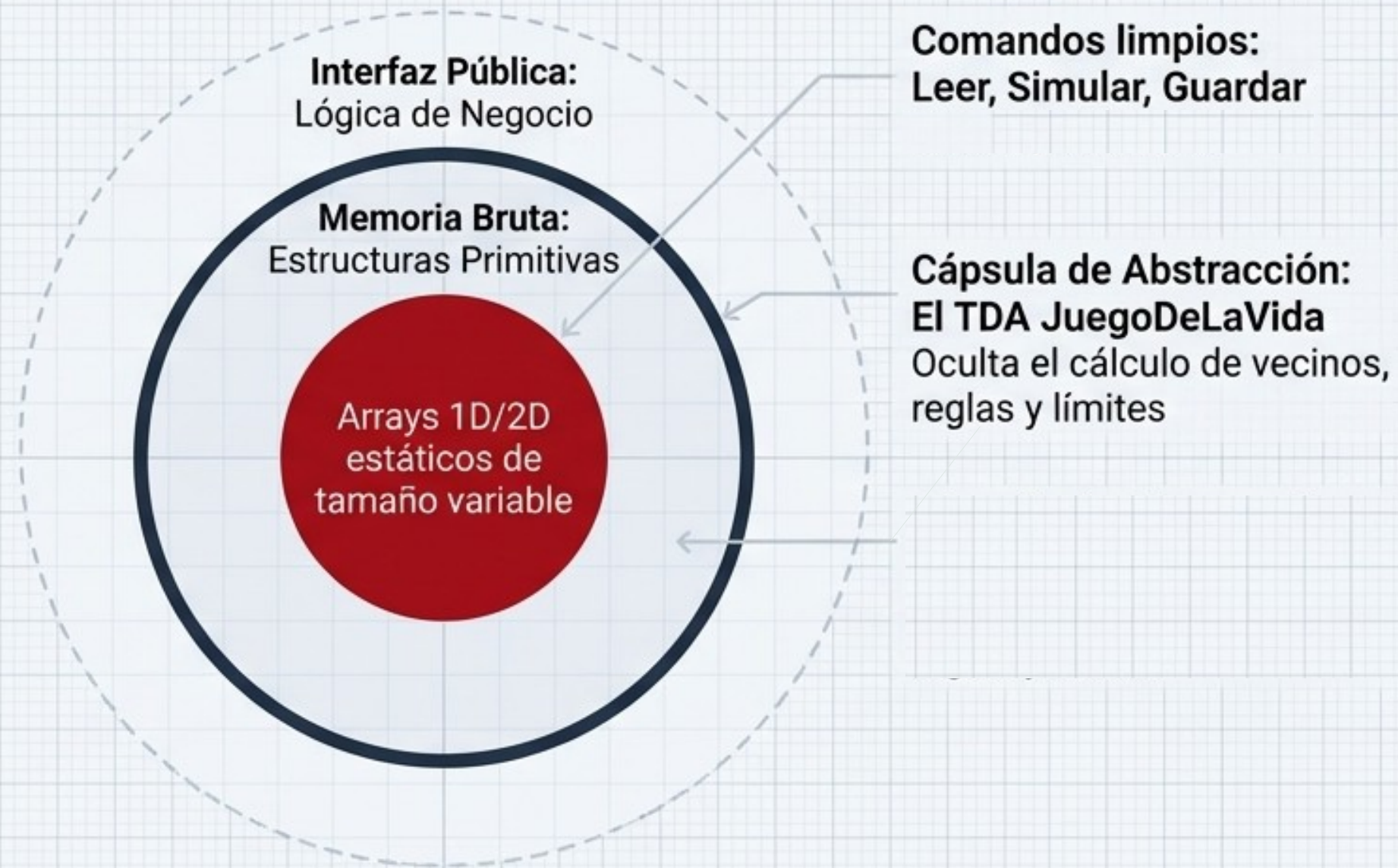
Especificar el TDA Juego de la vida indicando el tipo de dato y con qué propiedades funcionales lo dotaríais.



La idea no es que construyáis un programa que resuelva el juego. La idea es que especifiquéis el TDA con vistas a construir la clase correspondiente y resolver el juego.

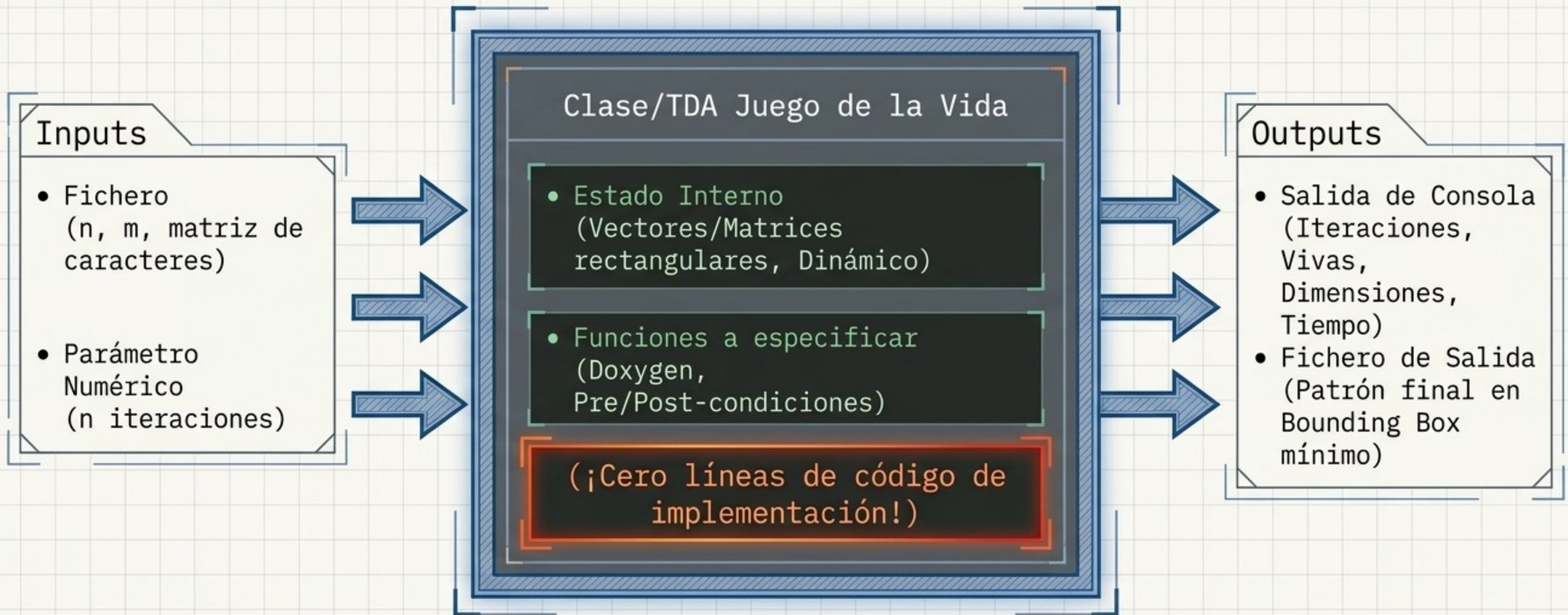
Todas las funciones que veáis interesantes para formar parte del TDA las habréis de especificar correctamente (preferiblemente usando Doxygen, aunque no es obligatorio) en el formato que queráis (precondiciones/postcondiciones, o con cláusulas de argumentos, valores de retorno, etc..).

Arquitectura de la Solución: El TDA



“La complejidad de reasignar matrices y rastrear rectángulos mínimos se oculta completamente detrás de llamadas abstractas limpias. El usuario ve un universo infinito; el sistema gestiona una matriz dinámica.”

Arquitectura de la Solución: Abstracción Requerida



Especificación de Estado: TDA JuegoDeLaVida

Clase: JuegoDeLaVida

Sección Privada (Estado de Memoria)

- `matriz_estado`: Array 2D (o 1D mapeado) de booleanos/enteros
- `filas`: Entero (Dimensión n actual)
- `columnas`: Entero (Dimensión m actual)
- `iteraciones_actuales`: Entero (Contador)

Sección Pública (Contrato Funcional)

Constructores (Inicialización por defecto y por fichero)

Operaciones I/O (`cargarPatron`, `guardarPatron`)

Lógica de Simulación (`simular`)

Consultores de Estado (`obtenerNumeroCeldasVivas`, `obtenerDimensiones`)

Interfaz TDA | I/O y Construcción

Firma: `cargarPatron(String nombreFichero)`

Descripción:

Inicializa el universo a partir de un fichero de texto con el patrón base.

Contrato de Ejecución:

[Precondición]: nombreFichero debe apuntar a un archivo de texto existente y legible. El archivo debe cumplir el formato estandarizado (n, m, matriz de "." y 'X').

[Postcondición]: El estado interno del TDA refleja el patrón del archivo. Las dimensiones internas filas y columnas se ajustan exactamente a n y m.

[Gestión de Error]: Excepción o estado nulo si el archivo es inválido o corrupto.

Interfaz TDA | Motor de Simulación

Firma: `simular(Entero n_iteraciones)`

Descripción:

Avanza el estado del universo aplicando las reglas de Conway durante un número definido de épocas.

Contrato de Ejecución:

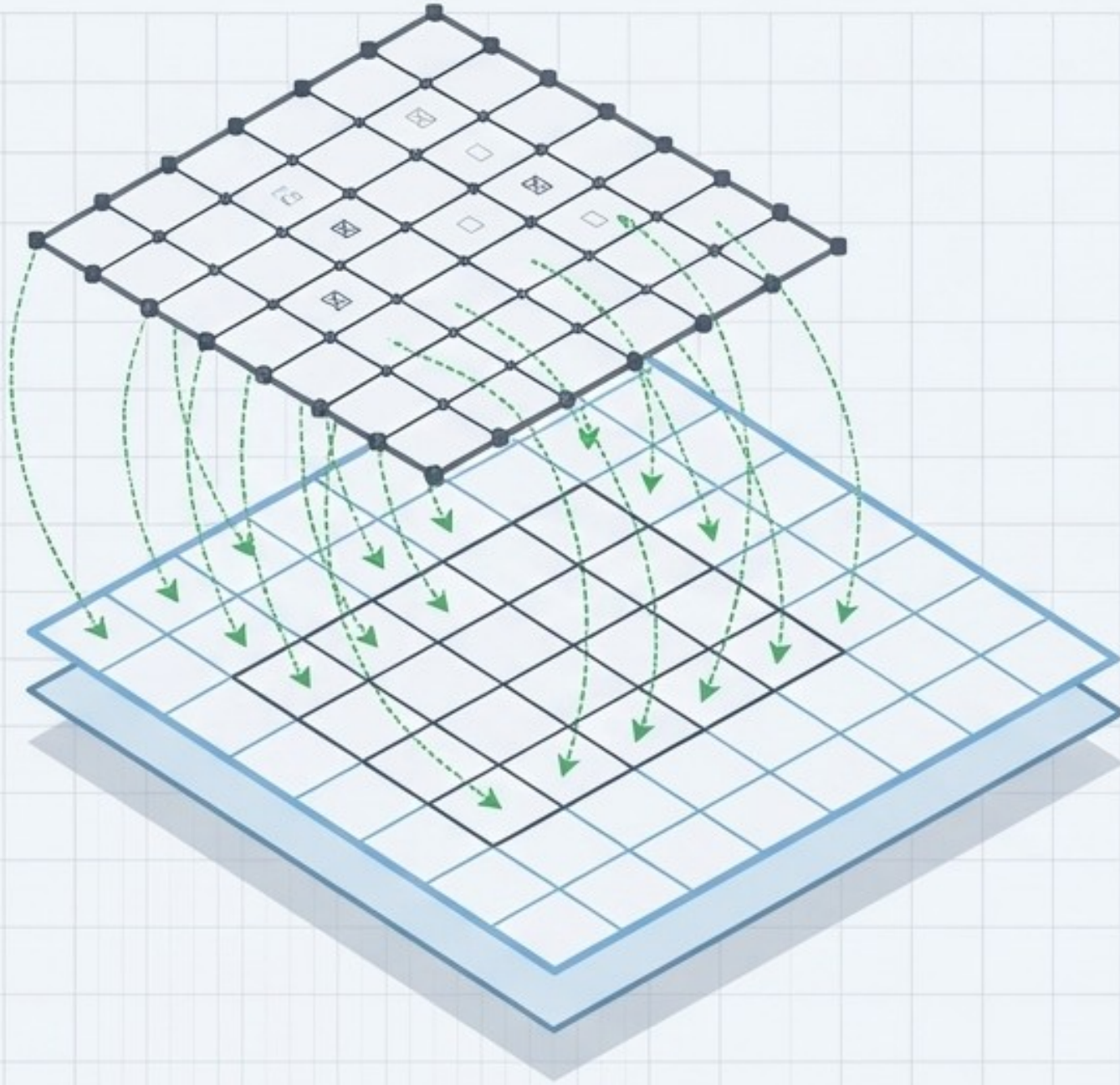
[Precondición]: El TDA debe haber sido inicializado con un patrón válido.
 $n_iteraciones > 0$.

[Postcondición]: El estado interno representa el universo tras n pasos temporales. El rectángulo mínimo ha sido recalculado (expandido o contraído según necesidad).

Proceso Abstracto (Por Iteración):

Calcular matriz de transición → Aplicar supervivencia/muerte → Redimensionar fronteras

Lógica Interna | Reasignación Dinámica



Firma Privada: `redimensionar(Entero nueva_f, Entero nueva_c)`

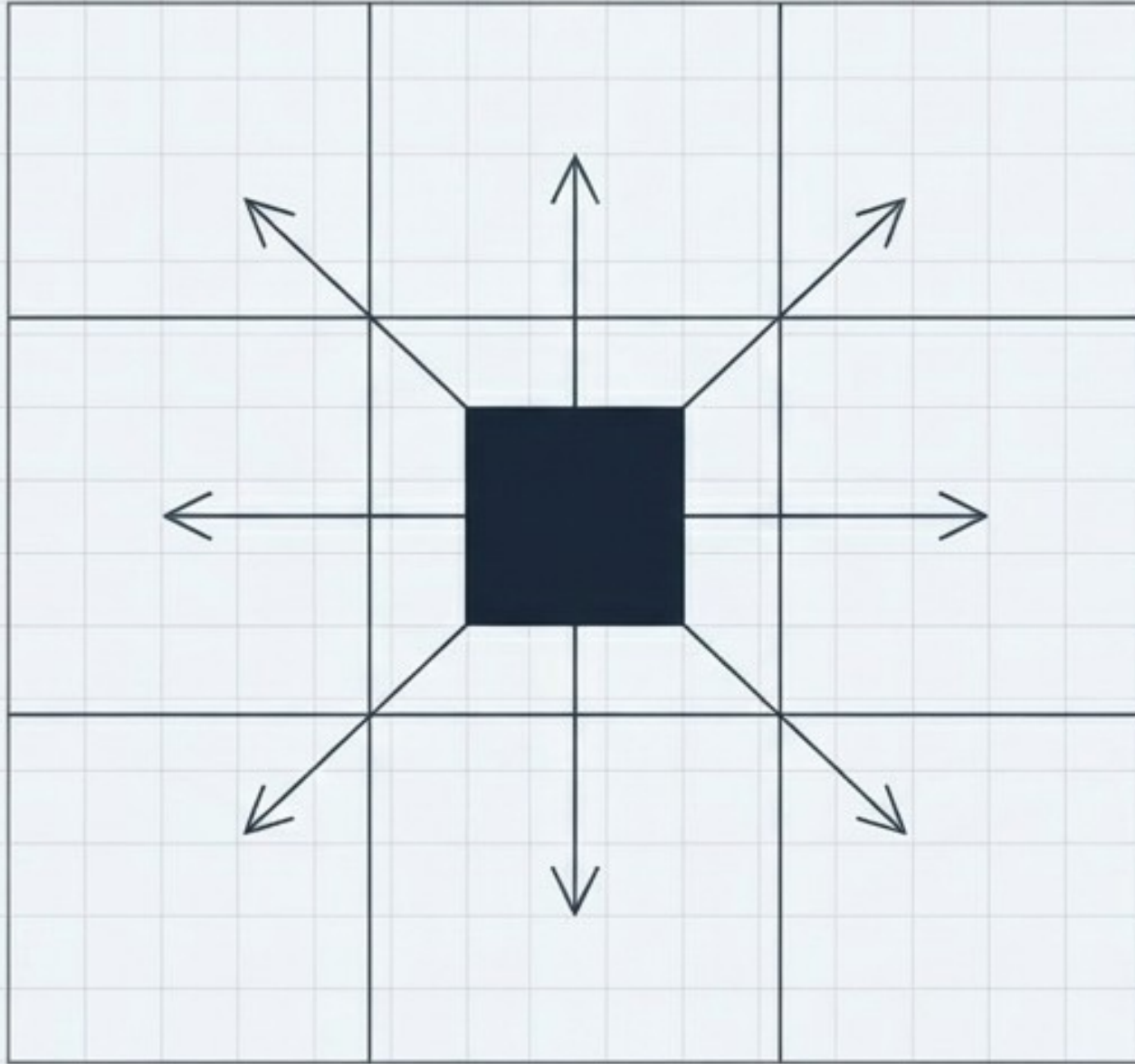
Descripción: Gestión transparente de la memoria primitiva.

Estrategia de Abstracción:

Cuando las celdas vivas tocan los bordes actuales en la iteración i , se reserva una nueva matriz rectangular más grande para $i+1$.

Se copia el estado anterior con un offset (desplazamiento) hacia el centro, y se libera la matriz original. Esto simula crecimiento infinito usando arrays fijos estáticos temporales, sin que el usuario lo perciba.

Lógica Interna | Evaluación de Estado



Firma Privada: `contarVecinosVivos(Entero f, Entero c) → Entero`

Descripción: Analiza la vecindad de Moore (8 direcciones) para una coordenada específica.

Contrato de Ejecución:

[Precondición]: Las coordenadas *f* (fila) y *c* (columna) deben pertenecer a los límites actuales de la matriz de memoria.

[Retorno]: Un valor entero estricto entre 0 y 8.

Nota Arquitectónica:

Las celdas fuera del rango de la matriz actual se consideran virtualmente 'muertas' (0), simplificando drásticamente el cálculo aritmético en los bordes de la matriz.

Interfaz TDA | Persistencia y Estadísticas

9

36

```
.....X.....
.....X.X.....
.....XX.....XX.....XX
.....X...X...XX.....XX
XX.....X...X...XX.....
XX.....X...X.XX...X.X.....
.....X...X...X.....
.....X...X.....
.....XX.....
```

Módulo de Persistencia

Firma: guardarPatron(String nombreFichero)

[Precondición]: Patrón actual válido. Si la cadena es vacía, se omite silenciosamente el guardado.

[Postcondición]: Genera un archivo .txt con el formato exacto exigido por las restricciones (rectángulo mínimo garantizado).

Módulo de Métricas

Firma 1: obtenerNumeroCeldasVivas() -> Entero

Firma 2: obtenerDimensiones() -> Tupla(Entero, Entero)

Propósito Arquitectónico:

Proveer datos de solo lectura para el flujo de salida por terminal, garantizando que no se exponga ni se corrompa la matriz de estado interna.

Consideraciones Generales y Puntuación



1. Podrán hacerse equipos de un máximo de 3 personas para debatir la solución y esa solución que se envíe se valorará con una puntuación igual para cada uno de los 3 miembros del equipo.



2. Los 3 miembros del equipo deberán subir a prado el mismo fichero pdf con la solución. Se sugiere como nombre del fichero `reto1.pdf`. Al principio del fichero deberá constar el nombre de los miembros del equipo.



3. Todas las soluciones viables se puntuarán con 0.1 para la evaluación continua.



4. Me interesa analizar vuestra capacidad de abstracción, de forma habréis de prestar atención al diseño de las clases, no a su implementación. De hecho no quiero que implementéis ni una sola línea de código. Todo habrá de hacerse a nivel de especificación. No pretendo que le dediquéis mucho tiempo, solo el necesario para pensar a alto nivel en la solución.